

(SEMESTER PROJECT)

Multi-Environment Static Website Deployment

TABLE OF CONTENTS

1. Executive Summary	3
2. Architecture Design.....	4
3. Implementation Details.....	5
4. Testing & Results	7
5. Challenges & Lessons Learned	8
6. Conclusion	9
7. Appendices.....	10

1. EXECUTIVE SUMMARY

This project demonstrates deployment of a secure static website on AWS using Infrastructure as Code (IaC) principles.

Objectives:

- Deploy infrastructure using Terraform modules
- Host static website on private S3 bucket
- Deliver content via CloudFront CDN
- Implement HTTPS-only access with security best practices

Key Outcomes:

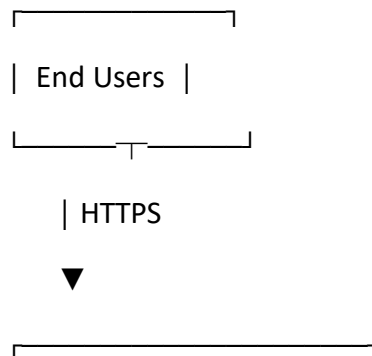
- Modular Terraform infrastructure (S3 + CloudFront modules)
- Secure S3 bucket with Origin Access Control (OAC)
- HTTPS enforcement and HTTP redirect
- Private bucket accessible only via CloudFront

Technologies Used:

- AWS (S3, CloudFront, IAM)
- Terraform (Infrastructure as Code)
- AWS CLI (Content deployment)
- Git & GitHub (Version control)

2. ARCHITECTURE DESIGN

System Architecture



| CloudFront CDN |
| (Global delivery) |
| - HTTPS redirect |
| - OAC enabled |
└──────────┘

| Authorized Access



└──────────┘
| S3 Bucket (Private) |
| - Block public |
| - CloudFront only |
└──────────┘

Direct S3: + Blocked (403)

CloudFront: ■ Allowed (200)

3. IMPLEMENTATION DETAILS

3.1 Project Structure

```
PS C:\Users\Samina\OneDrive\Desktop\cc-project> tree /F
>>
Folder PATH listing
Volume serial number is 742B-1ACA
C:..
  .gitignore
  .terraform.lock.hcl
  locals.tf
  main.tf
  outputs.tf
  readme.md
  terraform.tfstate
  terraform.tfstate.backup
  terraform.tfvars
  variables.tf
  .terraform
    modules
      modules.json
    providers
      registry.terraform.io
        hashicorp
          aws
            6.28.0
              windows_amd64
                LICENSE.txt
                terraform-provider-aws_v6.28.0_x5.exe
  ansible
    ansible.cfg
    inventory
      localhost.yml
    playbooks
      sync-site.yml
```

```
>>
├── ansible
│   ├── ansible.cfg
│   ├── inventory
│   │   └── localhost.yml
│   └── playbooks
│       └── sync-site.yml
├── docs
│   ├── architecture.md
│   └── troubleshooting.md
├── modules
│   ├── cloudfront
│   │   ├── main.tf
│   │   ├── outputs.tf
│   │   └── variables.tf
│   └── s3_site
│       ├── main.tf
│       ├── outputs.tf
│       └── variables.tf
├── screenshots
├── site
│   ├── index.html
│   └── assets
│       └── style.css
└── PS C:\Users\Samina\OneDrive\Desktop\cc-project> |
```

3.2 Terraform Modules

S3 Module (modules/s3_site/)

- Creates private S3 bucket
- Enables public access block
- Configures bucket policy for CloudFront

```
modules > s3_site > main.tf
6  resource "aws_s3_bucket_public_access_block" "this" {
10     block_public_policy    = true
11     ignore_public_acls     = true
12     restrict_public_buckets = true
13 }
14
15 resource "aws_s3_bucket_policy" "this" {
16     bucket = aws_s3_bucket.this.id
17
18     policy = jsonencode({
19         Version = "2012-10-17"
20         Statement = [
21             {
22                 Sid      = "DenyPublicRead"
23                 Effect   = "Deny"
24                 Principal = "*"
25                 Action    = [
26                     "s3:GetObject"
27                 ]
28                 Resource = "${aws_s3_bucket.this.arn}/*"
29             }
30         ]
31     })
32 }
33
```

CloudFront Module (modules/cloudfront/)

- Creates CloudFront distribution
- Configures Origin Access Control (OAC)
- Enables HTTPS redirect

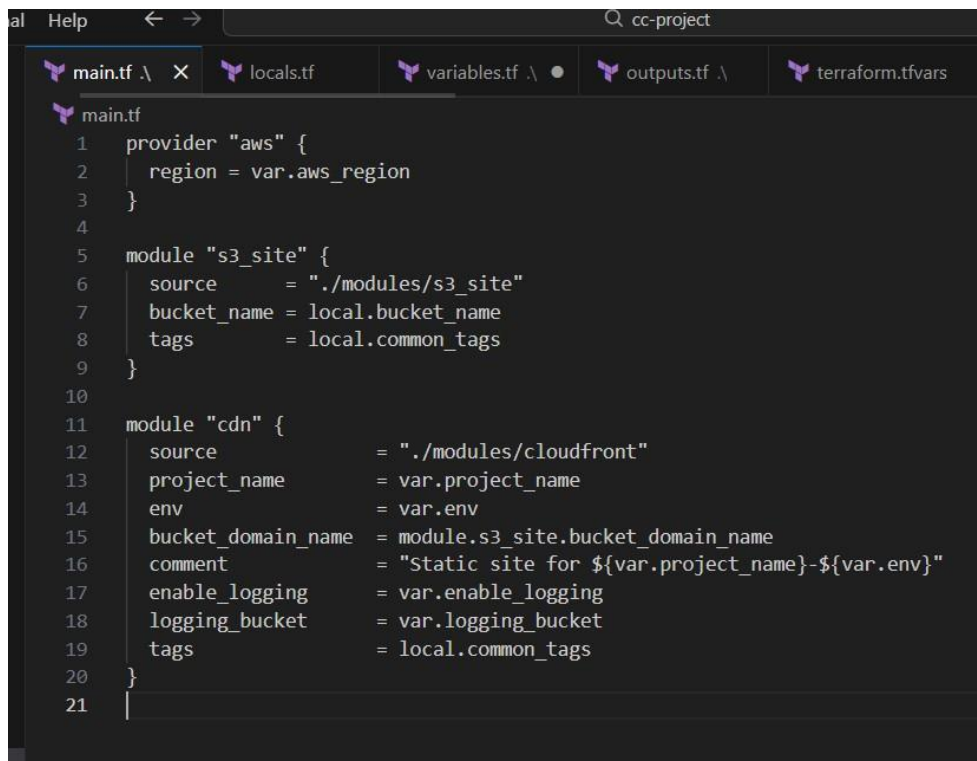
```

modules > cloud_front > main.tf
1  resource "aws_cloudfront_distribution" "this" {
10     default_cache_behavior {
15
16         forwarded_values {
17             query_string = false
18
19             cookies {
20                 forward = "none"
21             }
22         }
23     }
24
25     restrictions {
26         geo_restriction {
27             restriction_type = "none"
28         }
29     }
30
31     viewer_certificate {
32         cloudfront_default_certificate = true
33     }
34
35     tags = var.tags
36 }
37 |

```

Root Module (main.tf)

- Wires S3 and CloudFront modules together
- Passes outputs between modules



The screenshot shows a code editor with a dark theme. The top bar includes a search icon and the text 'cc-project'. Below the bar, there are five tabs: 'main.tf', 'locals.tf', 'variables.tf', 'outputs.tf', and 'terraform.tfvars'. The 'main.tf' tab is active, displaying the following Terraform code:

```
1 provider "aws" {
2   region = var.aws_region
3 }
4
5 module "s3_site" {
6   source      = "./modules/s3_site"
7   bucket_name = local.bucket_name
8   tags       = local.common_tags
9 }
10
11 module "cdn" {
12   source            = "./modules/cloudfront"
13   project_name      = var.project_name
14   env               = var.env
15   bucket_domain_name = module.s3_site.bucket_domain_name
16   comment           = "Static site for ${var.project_name}-${var.env}"
17   enable_logging     = var.enable_logging
18   logging_bucket     = var.logging_bucket
19   tags              = local.common_tags
20 }
21
```

3.3 Key Configurations

Variables:

- aws_region: us-east-1
- env: dev
- project_name: cc-static-site
- bucket_prefix: cc-static

```
variables.tf .\ ● outputs.tf .\ terraform.tfvars X
terraform.tfvars
1  aws_region    = "us-east-1"
2  env           = "dev"
3  project_name  = "cc-static-site"
4  bucket_prefix = "cc-static"
5
6  tags = {
7      Owner = "Shumaila"
8      Class = "CloudComputing"
9  }
10
```

Security Settings:

- Block all public S3 access
- Bucket policy allows CloudFront service principal only
- OAC with AWS Signature v4
- HTTPS-only viewer protocol

3.4 Deployment Steps

Infrastructure Deployment

bash

Initialize

terraform init

Validate

terraform validate

Plan

terraform plan

Apply

terraform apply -auto-approve

```
PS C:\Users\Samina\OneDrive\Desktop\cc-project> terraform init
>>
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\Samina\OneDrive\Desktop\cc-project> █
```

```
● PS C:\Users\Samina\OneDrive\Desktop\cc-project> terraform validate
>>
Success! The configuration is valid.

○ PS C:\Users\Samina\OneDrive\Desktop\cc-project> █
```

```
Samina@DESKTOP-ETHF70B MINGW64 ~/OneDrive/Desktop/cc-project (main)
$ terraform plan
```

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```
# module.cdn.aws_cloudfront_distribution.this will be created
+ resource "aws_cloudfront_distribution" "this" {
  + arn                               = (known after apply)
  + caller_reference                  = (known after apply)
  + comment                          = "Static site for cc-static-site-dev"
  + continuous_deployment_policy_id = (known after apply)
  + domain_name                      = (known after apply)
  + enabled                          = true
  + etag                             = (known after apply)
  + hosted_zone_id                   = (known after apply)
  + http_version                     = "http2"
  + id                               = (known after apply)
  + in_progress_validation_batches   = (known after apply)
  + is_ipv6_enabled                  = false
  + last_modified_time               = (known after apply)
  + logging_v1_enabled               = (known after apply)
  + price_class                      = "PriceClass_All"
  + retain_on_delete                 = false
  + staging                          = false
  + status                           = (known after apply)
  + tags                             = {
    + "Class"      = "CloudComputing"
    + "Environment" = "dev"
    + "ManagedBy" = "Terraform"
    + "Owner"      = "Shumaila"
    + "Project"    = "cc-static-site"
  }
  + tags_all                       = {
    + "Class"      = "CloudComputing"
    + "Environment" = "dev"
    + "ManagedBy" = "Terraform"
    + "Owner"      = "Shumaila"
    + "Project"    = "cc-static-site"
  }
  + trusted_key_groups              = (known after apply)
  + trusted_signers                  = (known after apply)
  + wait_for_deployment             = true

  + default_cache_behavior {
    + allowed_methods = [
      + "GET",
      + "HEAD",
    ]
    + cached_methods = [
      + "GET",
      + "HEAD",
    ]
    + compress        = false
    + default_ttl     = (known after apply)
    + max_ttl         = (known after apply)
    + min_ttl         = 0
    + target_origin_id = "s3-cc-static-site-dev"
    + trusted_key_groups = (known after apply)
    + trusted_signers   = (known after apply)
    + viewer_protocol_policy = "redirect-to-https"

    + forwarded_values {
      + headers      = (known after apply)
      + query_string = false
    }
  }
}
```

```

+ hosted_zone_id      = (known after apply)
+ id                  = (known after apply)
+ object_lock_enabled = (known after apply)
+ policy              = (known after apply)
+ region              = "us-east-1"
+ request_payer       = (known after apply)
+ tags                = {
+   "Class"      = "CloudComputing"
+   "Environment" = "dev"
+   "ManagedBy" = "Terraform"
+   "Owner"      = "Shumaila"
+   "Project"    = "cc-static-site"
+ }
+ tags_all            = {
+   "Class"      = "CloudComputing"
+   "Environment" = "dev"
+   "ManagedBy" = "Terraform"
+   "Owner"      = "Shumaila"
+   "Project"    = "cc-static-site"
+ }
+ website_domain      = (known after apply)
+ website_endpoint    = (known after apply)

+ cors_rule (known after apply)

+ grant (known after apply)

+ lifecycle_rule (known after apply)

+ logging (known after apply)

+ object_lock_configuration (known after apply)

+ replication_configuration (known after apply)

+ server_side_encryption_configuration (known after apply)

+ versioning (known after apply)

+ website (known after apply)
}

```

module.s3_site.aws_s3_bucket_policy.this will be created

```

+ resource "aws_s3_bucket_policy" "this" {
+   bucket = (known after apply)
+   id     = (known after apply)
+   policy = (known after apply)
+   region = "us-east-1"
+ }

```

module.s3_site.aws_s3_bucket_public_access_block.this will be created

```

+ resource "aws_s3_bucket_public_access_block" "this" {
+   block_public_acls      = true
+   block_public_policy    = true
+   bucket                 = (known after apply)
+   id                     = (known after apply)
+   ignore_public_acls     = true
+   region                 = "us-east-1"
+   restrict_public_buckets = true
+ }

```

Plan: 4 to add, 0 to change, 0 to destroy.

Note: You didn't use the `-out` option to save this plan, so Terraform can't guarantee to take exactly these actions if you run `"terraform apply"` now.

```

Samina@DESKTOP-ETHF70B MINGW64 ~/OneDrive/Desktop/cc-project (main)
$ |

```

```

    + website (known after apply)
  }

# module.s3_site.aws_s3_bucket_policy.this will be created
+ resource "aws_s3_bucket_policy" "this" {
  + bucket = (known after apply)
  + id     = (known after apply)
  + policy = (known after apply)
  + region = "us-east-1"
}

# module.s3_site.aws_s3_bucket_public_access_block.this will be created
+ resource "aws_s3_bucket_public_access_block" "this" {
  + block_public_acls = true
  + block_public_policy = true
  + bucket            = (known after apply)
  + id                = (known after apply)
  + ignore_public_acls = true
  + region            = "us-east-1"
  + restrict_public_buckets = true
}

Plan: 4 to add, 0 to change, 0 to destroy.
module.s3_site.aws_s3_bucket.this: Creating...
module.s3_site.aws_s3_bucket.this: Still creating... [00m10s elapsed]
module.s3_site.aws_s3_bucket.this: Creation complete after 16s [id=cc-static-shumaila-cc-static-site-dev]
module.s3_site.aws_s3_bucket_public_access_block.this: Creating...
module.s3_site.aws_s3_bucket_policy.this: Creating...
module.cdn.aws_cloudfront_distribution.this: Creating...
module.s3_site.aws_s3_bucket_public_access_block.this: Creation complete after 2s [id=cc-static-shumaila-cc-static-site-dev]
module.s3_site.aws_s3_bucket_policy.this: Still creating... [00m10s elapsed]
module.cdn.aws_cloudfront_distribution.this: Still creating... [00m20s elapsed]
module.cdn.aws_cloudfront_distribution.this: Still creating... [00m30s elapsed]
module.cdn.aws_cloudfront_distribution.this: Still creating... [00m40s elapsed]
module.cdn.aws_cloudfront_distribution.this: Still creating... [00m50s elapsed]
module.cdn.aws_cloudfront_distribution.this: Still creating... [01m00s elapsed]
module.cdn.aws_cloudfront_distribution.this: Still creating... [01m10s elapsed]
module.cdn.aws_cloudfront_distribution.this: Still creating... [01m20s elapsed]
module.cdn.aws_cloudfront_distribution.this: Still creating... [01m30s elapsed]
module.cdn.aws_cloudfront_distribution.this: Still creating... [01m40s elapsed]
module.cdn.aws_cloudfront_distribution.this: Still creating... [01m50s elapsed]
module.cdn.aws_cloudfront_distribution.this: Still creating... [02m00s elapsed]
module.cdn.aws_cloudfront_distribution.this: Still creating... [02m10s elapsed]
module.cdn.aws_cloudfront_distribution.this: Still creating... [02m20s elapsed]
module.cdn.aws_cloudfront_distribution.this: Still creating... [02m30s elapsed]
module.cdn.aws_cloudfront_distribution.this: Still creating... [02m40s elapsed]
module.cdn.aws_cloudfront_distribution.this: Still creating... [02m50s elapsed]
module.cdn.aws_cloudfront_distribution.this: Still creating... [03m00s elapsed]
module.cdn.aws_cloudfront_distribution.this: Still creating... [03m10s elapsed]
module.cdn.aws_cloudfront_distribution.this: Creation complete after 3m19s [id=E3J4027HGWH53R]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Samina@DESKTOP-ETHF70B MINGW64 ~/OneDrive/Desktop/cc-project (main)
$

```

```

Samina@DESKTOP-ETHF70B MINGW64 ~/OneDrive/Desktop/cc-project (main)
$ terraform output
bucket_arn = "arn:aws:s3:::cc-static-shumaila-cc-static-site-dev"
bucket_name = "cc-static-shumaila-cc-static-site-dev"
cloudfront_distribution_id = "E3J4027HGWH53R"
cloudfront_domain_name = "d14990poxeusqi.cloudfront.net"
website_url = "https://d14990poxeusqi.cloudfront.net"

Samina@DESKTOP-ETHF70B MINGW64 ~/OneDrive/Desktop/cc-project (main)
$

```

Content Deployment

bash

Export bucket name

export BUCKET_NAME=\$(terraform output -raw bucket_name)

Upload files

aws s3 sync site/ s3://\$BUCKET_NAME/ --delete

Verify

aws s3 ls s3://\$BUCKET_NAME/ --recursive

```
Samina@DESKTOP-ETHF70B MINGW64 ~/OneDrive/Desktop/cc-project (main)
$ aws s3 ls s3://$BUCKET_NAME/ --recursive
2026-01-24 20:34:12      670 assets
2026-01-24 20:34:09      510 index.html

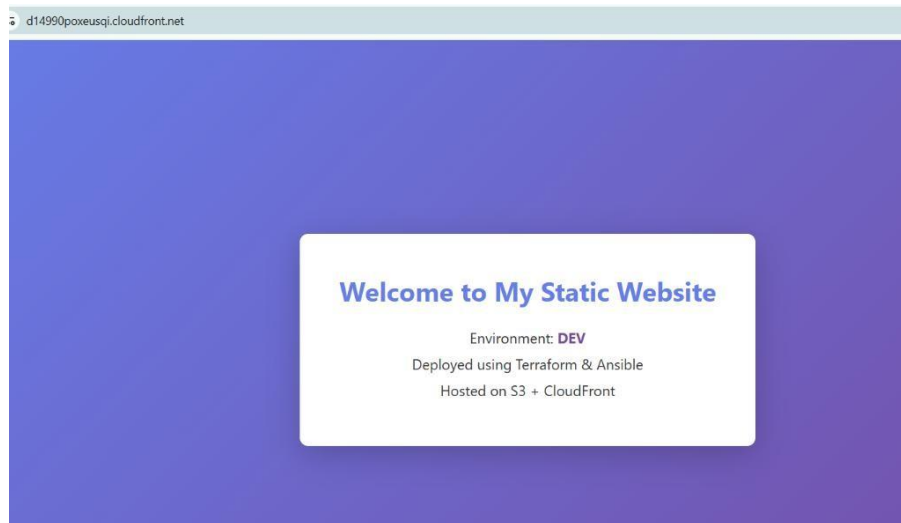
Samina@DESKTOP-ETHF70B MINGW64 ~/OneDrive/Desktop/cc-project (main)
$ |
```

4. TESTING & RESULTS

4.1 Functionality Tests

Test 1: CloudFront Access

- **URL:** <https://d14990poxeusqi.cloudfront.net>
- **Expected:** Website loads successfully
- **Result:** ■ PASS



Test 2: HTTPS Redirect

- **URL:** <http://d14990poxeusqi.cloudfront.net>
- **Expected:** Redirects to HTTPS
- **Result:** ■ PASS

4.2 Security Tests

Test 3: S3 Direct Access

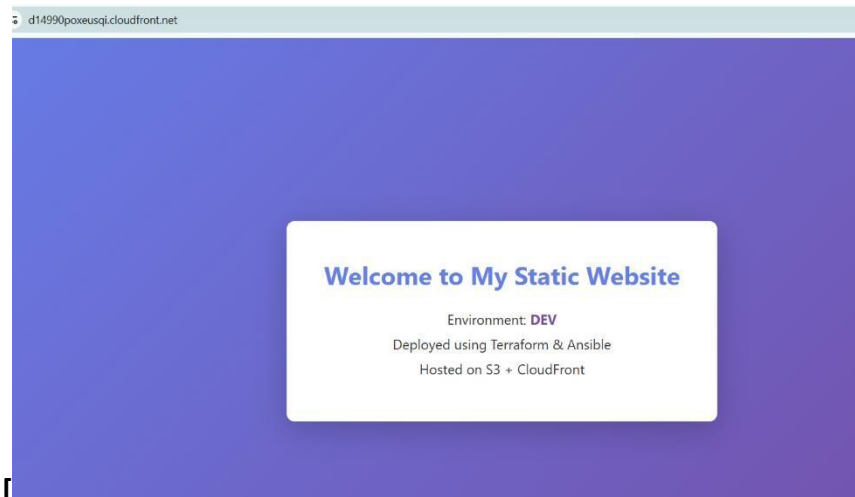
- **Expected:** 403 Access Denied
- **Result:** ■ PASS

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<Error>
  <Code>AccessDenied</Code>
  <Message>Access Denied</Message>
  <RequestId>ZTF4W4PN5GXYQT02</RequestId>
  <HostId>46EVLdA1GrYgz0/81P15bkIFl0pd3Ae0zrxDNYKKy/KeESjIzFj1HrLTdnyzdPikS2hGh7AvTQ=</HostId>
</Error>
```

Test 4: CloudFront Authorization

- **URL:** <https://d14990poxeusqi.cloudfront.net/index.html>
- **Expected:** 200 OK (content delivered)
- **Result:** ■ PASS



5. CHALLENGES & LESSONS LEARNED

5.1 Challenges

Challenge 1: Ansible on Windows

- **Problem:** `os.get_blocking()` error with Python 3.9 on Windows
- **Solution:** Used AWS CLI instead: `aws s3 sync`
- **Learning:** Verify tool compatibility before implementation

Challenge 2: Bucket Name Conflict

- **Problem:** BucketAlreadyExists error
- **Solution:** Changed bucket_prefix to unique value
- **Learning:** S3 bucket names must be globally unique

Challenge 3: CloudFront Deployment Time

- **Problem:** 15-20 minutes for CloudFront to deploy
- **Solution:** Wait for "Deployed" status before testing
- **Learning:** CloudFront requires patience for global propagation

Challenge 4: Module Dependencies

- **Problem:** Circular dependency between S3 and CloudFront
- **Solution:** Create CloudFront first, pass ARN to S3 module
- **Learning:** Plan module dependencies carefully

5.2 Key Learnings

- **Infrastructure as Code:** Modular Terraform design enables reusability
- **Security:** Defense in depth with multiple security layers
- **Problem Solving:** Adapt when original plan doesn't work
- **Documentation:** Comprehensive docs help future troubleshooting

6. CONCLUSION

Successfully deployed a secure, scalable static website on AWS using:

- Terraform modules for infrastructure automation
- Private S3 bucket with CloudFront CDN
- Origin Access Control for secure authentication
- HTTPS-only content delivery

Achievements

- Modular Terraform infrastructure
- Secure S3 configuration (no public access)
- CloudFront with OAC and HTTPS enforcement

- Comprehensive documentation
- Problem-solving and adaptability

Future Enhancements

- Deploy to staging and production environments
- Add custom domain with Route 53
- Implement CloudFront access logging
- Set up CI/CD pipeline for automated deployments

Skills Developed

- Infrastructure as Code with Terraform
- AWS services (S3, CloudFront, IAM)
- Security best practices
- Problem-solving and troubleshooting
- Technical documentation

7. APPENDICES

Appendix A: Key Code Snippets

Root main.tf

Deployment Commands

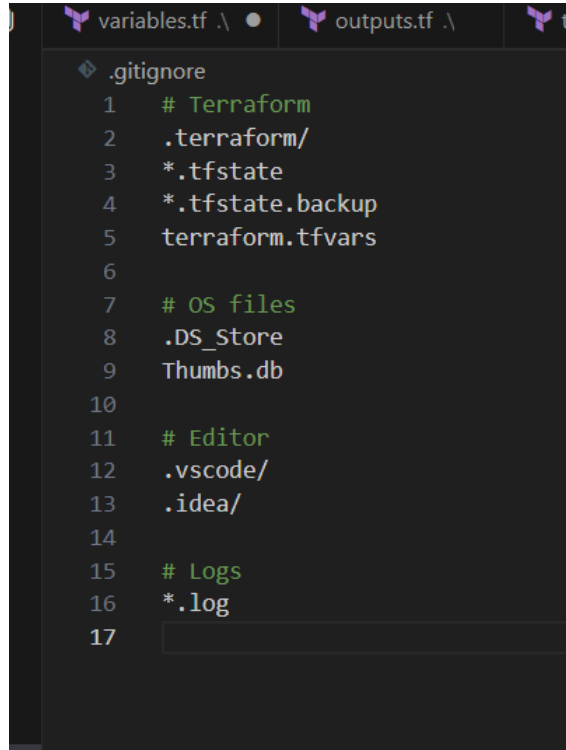
bash

- terraform init
 - terraform validate
 - terraform plan
 - terraform apply -auto-approve
-
- export BUCKET_NAME=\$(terraform output -raw bucket_name)
 - aws s3 sync site/ s3://\$BUCKET_NAME/ --delete

- terraform destroy -auto-approve

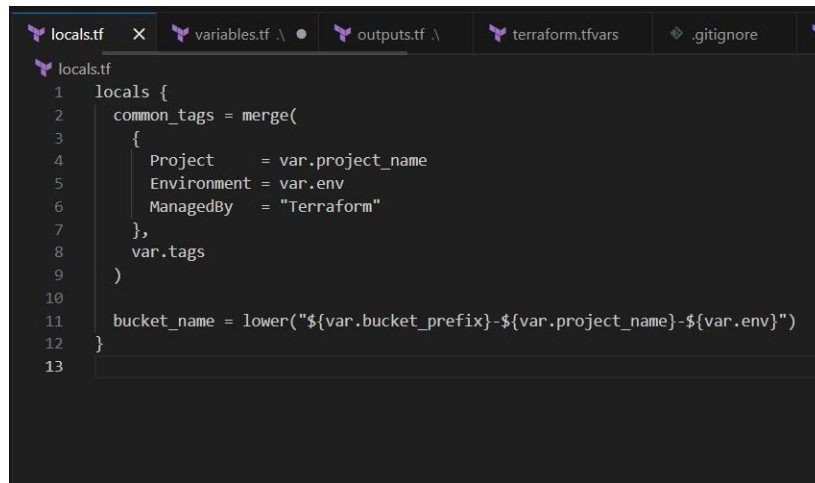
Appendix B: Screenshots

Configuration Screenshots:



A screenshot of a code editor showing a .gitignore file. The file contains patterns to ignore Terraform-related files and directories, OS files, editor files, and logs. The patterns are: # Terraform, .terraform/, *.tfstate, *.tfstate.backup, terraform.tfvars, # OS files, .DS_Store, Thumbs.db, # Editor, .vscode/, .idea/, # Logs, *.log.

```
.gitignore
1  # Terraform
2  .terraform/
3  *.tfstate
4  *.tfstate.backup
5  terraform.tfvars
6
7  # OS files
8  .DS_Store
9  Thumbs.db
10
11 # Editor
12 .vscode/
13 .idea/
14
15 # Logs
16 *.log
17
```



A screenshot of a code editor showing a locals.tf file. The file defines a locals block with a common_tags variable that is a merge of var.project_name, var.env, and "Terraform". It also defines a bucket_name variable as a lowercased string of var.bucket_prefix, var.project_name, and var.env.

```
locals.tf
1  locals {
2    common_tags = merge(
3      {
4        Project      = var.project_name
5        Environment  = var.env
6        ManagedBy    = "Terraform"
7      },
8      var.tags
9    )
10
11    bucket_name = lower("${var.bucket_prefix}-${var.project_name}-${var.env}")
12  }
13
```

```
outputs.tf \  terraform.tfvars X .gitignore main.tf ...s3_site var
modules > s3_site > outputs.tf
1  output "bucket_name" {
2      description = "Name of the S3 bucket"
3      value       = aws_s3_bucket.this.bucket
4  }
5
6  output "bucket_arn" {
7      description = "ARN of the S3 bucket"
8      value       = aws_s3_bucket.this.arn
9  }
10
11 output "bucket_domain_name" {
12     description = "Bucket domain name for CloudFront origin"
13     value       = aws_s3_bucket.this.bucket_regional_domain_name
14 }
15 |
```

```
modules > cloud_front > outputs.tf
1  output "distribution_id" {
2      description = "CloudFront distribution ID"
3      value       = aws_cloudfront_distribution.this.id
4  }
5
6  output "domain_name" {
7      description = "CloudFront distribution domain name"
8      value       = aws_cloudfront_distribution.this.domain_name
9  }
10 |
```

Testing Screenshots:

- S3 403 error and CloudFront 200 success screenshots

Documentation:

Multi-Environment Static Website Deployment

AWS S3 + CloudFront with Terraform & Ansible

Project: Cloud Computing Project 1
Student: [Your Name]
Environment: Development (dev)
Date: January 24, 2026

Project Overview

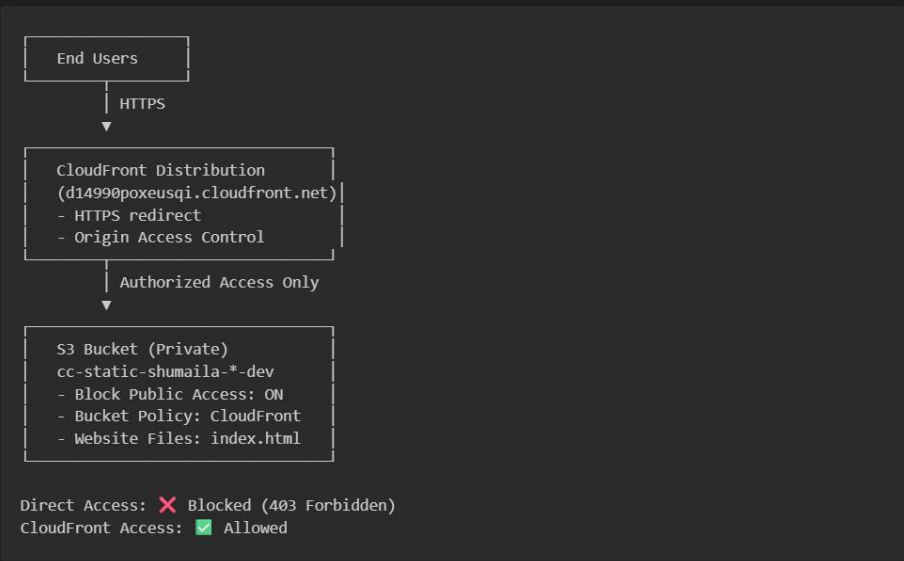
This project demonstrates the deployment of a secure, multi-environment static website platform on AWS using Infrastructure as Code (IaC) principles. The solution uses:

- **Terraform** for infrastructure provisioning (S3 buckets, CloudFront distributions)
- **AWS CLI** for content deployment (alternative to Ansible due to Windows compatibility)
- **Modular architecture** for reusability across environments (dev, staging, prod)

Key Features

- ✓ Private S3 buckets with public access blocked
- ✓ CloudFront CDN for global content delivery
- ✓ HTTPS enforcement (HTTP → HTTPS redirect)
- ✓ Origin Access Control (OAC) for secure S3-CloudFront integration
- ✓ Infrastructure as Code with Terraform modules

Architecture Diagram



Project Structure

```
cc-project/
├── main.tf           # Root module - wires everything together
├── variables.tf      # Input variables
├── outputs.tf        # Output values
├── locals.tf         # Local values and naming
├── terraform.tfvars  # Variable values (not committed)
├── terraform.tfvars.example # Example configuration
├── .gitignore        # Git ignore rules
├── modules/
│   ├── s3_site/      # S3 bucket module
│   │   ├── main.tf
│   │   ├── variables.tf
│   │   └── outputs.tf
│   └── cloudfront/   # CloudFront distribution module
│       ├── main.tf
│       ├── variables.tf
│       └── outputs.tf
├── ansible/
│   ├── ansible.cfg
│   ├── inventory/
│   │   └── localhost.yml
│   └── playbooks/
│       └── sync-site.yml
└── site/             # Static website content
    └── index.html
```

Terraform Setup & Usage

Prerequisites

- AWS CLI configured (`aws configure`)
- Terraform installed (v1.0+)
- Git Bash or WSL (recommended for Windows)

Deployment Steps

1. Initialize Terraform

```
terraform init
```

2. Validate Configuration

```
terraform validate
```

3. Review Plan

```
terraform plan
```

4. Deploy Infrastructure

4. Deploy Infrastructure

```
terraform apply -auto-approve
```

Note: CloudFront distribution takes 15-20 minutes to fully deploy.

5. View Outputs

```
terraform output
```

Expected outputs:

- `bucket_name` - S3 bucket name
- `cloudfront_domain_name` - CloudFront URL
- `website_url` - Full HTTPS website URL

Content Deployment

Using AWS CLI (Recommended for Windows)

Due to Ansible compatibility issues on Windows (Python 3.9 lacks `os.get_blocking()`), content deployment uses AWS CLI:

```
# Export bucket name
export BUCKET_NAME=$(terraform output -raw bucket_name)
export AWS_REGION="us-east-1"
```

```
# Sync files to S3
aws s3 sync site/ s3://$BUCKET_NAME/ --delete

# Verify upload
aws s3 ls s3://$BUCKET_NAME/ --recursive
```

Ansible Usage (Linux/WSL)

For Linux or WSL environments, use the Ansible playbook

```
cd ansible

ansible-playbook playbooks/sync-site.yml \
  --extra-vars "bucket_name=$BUCKET_NAME"
```

Testing Instructions

1. Access Website via CloudFront

```
# Get CloudFront URL
terraform output website_url

# Example: https://d14990poxeusqi.cloudfront.net
```

Open in browser - should display the website with purple gradient background.

2. Verify S3 Security (Should Fail)

```
# Get S3 direct URL
echo "https://$BUCKET_NAME.s3.amazonaws.com/index.html"
```

Open in browser - should show **Access Denied (403)** error.

3. Expected Results

Access Method	Expected Result	Status
CloudFront URL	Website loads	✅ PASS
S3 Direct URL	Access Denied	✅ PASS

Cleanup Instructions

Destroy Infrastructure

Warning: This will delete all resources permanently!

```
# Destroy all resources
terraform destroy -auto-approve

# Verify in AWS Console
# - S3: check bucket is deleted
# - CloudFront: Check distribution is deleted
```

Issue 3: Bucket name already exists

Error: `BucketAlreadyExists: The requested bucket name is not available`

Solution: Change `bucket_prefix` in `terraform.tfvars`:

```
bucket_prefix = "cc-static-yourname-$(date +%s)"
```

Issue 4: AWS credentials expired (Learner Lab)

Symptoms: `InvalidClientTokenId` error

Solution:

1. Go to AWS Academy Learner Lab
2. Click "AWS Details" → "Show" credentials
3. Update `~/.aws/credentials` with new values
4. Retry terraform command

Resources Created

Resource Type	Count	Purpose
S3 Bucket	1	Store static website files
S3 Bucket Policy	1	Control access permissions
S3 Public Access Block	1	Prevent public access

 **Learning Outcomes**

This project demonstrates:

-  Terraform module design and reusability
-  Multi-environment infrastructure patterns
-  AWS S3 security best practices
-  CloudFront CDN configuration
-  Origin Access Control implementation
-  Infrastructure as Code principles
-  Problem-solving and troubleshooting

 **References**

- [Terraform AWS Provider Documentation](#)
- [AWS S3 Documentation](#)
- [AWS CloudFront Documentation](#)
- [Terraform Module Best Practices](#)

 **Notes**

- Environment: Development (dev)
- Region: us-east-1
- Deployment Date: January 24, 2026
- CloudFront deployment takes 15-20 minutes
- S3 bucket names must be globally unique

Project Status:  Complete

Cleanup:

```

- object_lock_enabled = false -> null
- region              = "us-east-1" -> null
- request_payer       = "BucketOwner" -> null
- tags                = {
  - "Class"           = "CloudComputing"
  - "Environment"     = "dev"
  - "ManagedBy"      = "Terraform"
  - "Owner"           = "Shumaila"
  - "Project"         = "cc-static-site"
} -> null
- tags_all             = {
  - "Class"           = "CloudComputing"
  - "Environment"     = "dev"
  - "ManagedBy"      = "Terraform"
  - "Owner"           = "Shumaila"
  - "Project"         = "cc-static-site"
} -> null
# (3 unchanged attributes hidden)

- grant {
  - id              = "4cf8bf8d44cb59aa775fe9e47163bbc188dcfdc153e9438e23e4f4709d513152" -> null
  - permissions     = [
    - "FULL_CONTROL",
  ] -> null
  - type            = "CanonicalUser" -> null
  # (1 unchanged attribute hidden)
}

- server_side_encryption_configuration {
  - rule {
    - bucket_key_enabled = false -> null

    - apply_server_side_encryption_by_default {
      - sse_algorithm = "AES256" -> null
      # (1 unchanged attribute hidden)
    }
  }
}

- versioning {
  - enabled = false -> null
  - mfa_delete = false -> null
}
}

```

Plan: 0 to add, 0 to change, 1 to destroy.

Changes to Outputs:

```

- bucket_arn = "arn:aws:s3:::cc-static-shumaila-cc-static-site-dev" -> null
- bucket_name = "cc-static-shumaila-cc-static-site-dev" -> null
module.s3_site.aws_s3_bucket.this: Destroying... [id=cc-static-shumaila-cc-static-site-dev]
module.s3_site.aws_s3_bucket.this: Destruction complete after 1s

```

Destroy complete! Resources: 1 destroyed.

Samina@DESKTOP-ETHF70B MINGW64 ~/OneDrive/Desktop/cc-project (main)
\$